

Exercise 1: Transparent Classroom Voting System (Voting Contract)

1. Building Voting Logic (Voting Logic)

Smart Contract in this situation acts like a class in object-oriented programming, storing State variables such as the number of votes and voting status of each address, along with Functions such as the voting function.

Processing flow when student A sends the command "Vote for Binh":

Step 1 (Validation / Check):

- Smart Contract will check its internal state, specifically checking the State variables to determine whether the wallet address of A (transaction sender) has been marked as voted or not.
- If A's wallet address has been previously voted, the transaction will be rejected and execution will be stopped.
- This process ensures that each voter can only cast exactly 1 vote, following the predetermined logic of the contract.

Step 2 (Record / Effect):

- If the check in Step 1 confirms that A's wallet address has never voted, the contract will execute the Voting Functions.
- Vote count variable of candidate Binh will be updated and increased by 1 unit (change in status/data of the contract).

Step 3 (Status Lock / Effect & Security):

- To ensure that A cannot vote for Chi again, in the same transaction (before the transaction is confirmed and recorded in the Blockchain), Smart Contract must update A's status.
- A's wallet address will be marked as "voted" in the contract's State variables.
- Since the Smart Contract is deterministic—the same input always produces the same output regardless of the node—any subsequent voting transaction from A will be immediately blocked in Step 1.

2. Transparency and Immutability

Based on the principle of Smart Contract, the contract creator (Admin) cannot change the election result from "Very won" to "An won" after the voting and calculation process has taken place.

The reason is:

1. "Code is Law" principle: In the Blockchain environment, the source code (Code) is the law. Smart Contracts are designed so that the execution results of the code are ultimate and irreversible. The encrypted vote calculation results cannot be changed by the Admin's personal will.
2. Immutability: Smart Contracts, after being deployed on the Blockchain, are usually immutable; code cannot be modified. This ensures that the originally defined voting rules remain intact and the results are publicly calculated, preventing anyone (including its creator) from cheating or tampering with votes.
3. Execution mechanism: Smart contracts operate on the Trustless principle, without the need for a third party (such as Admin) to ensure execution. Electoral contracts rely on "cryptographic enforcement" to ensure fairness, not on legal enforcement or human trust.

The use of Voting Contract on Blockchain aims to make the election process transparent and eliminate fraud by publicly calculating results.

Exercise 2: Startup group's common "safe" (Multi-signature model - Multisig)

1. Contract creation

1. Save the list of "Owners" (Owners):

- The smart contract only identifies members A, B, C through their wallet addresses, not their real-life names.
- When writing code, the Contract will store this list of wallet addresses in its Data (State variables). This contract stores the list of owners and counts the number of confirmations needed.

2. "Required Signatures" variable:

- According to the rules established in the context, to transfer money, confirmation from at least 2 out of 3 members is required.
- Therefore, the variable "Confirmation Threshold" will be set to 2.

2. Simulate the Withdrawal process (Transaction Lifecycle)

Smart Contract is a computer program that automatically executes when predetermined conditions are met.

- Initialization:

- Member A creates a proposal "Transfer 10 ETH to pay office rent"
- At this time, the status of the proposal is Pending.
- The current number of positive votes is 1 (because A is the one who initiated and confirmed the proposal). The transaction has been saved but not yet executed.

- Consensus:

- Member B checks the proposal and calls the confirmTransaction function.
- The number of votes in favor changed from 1 to 2.

- Implementation:

- As soon as the number of positive votes reaches the confirmation threshold (2/3), the Smart Contract will automatically perform a transaction to transfer 10 ETH out of the shared wallet.
- The status of the proposal is now Executed.

3. Risk management perspective

1. What happens if member C loses his private key?

- Groups A and B can still withdraw money.
- Since the confirmation threshold is only 2 (2/3), A and B only need to agree and confirm the transaction to meet the required threshold.

2. What happens if both B and C lose their keys?

- The amount in the contract will be locked forever as the 2-vote consensus threshold is never reached.
- Only member A can confirm (1 vote), but the threshold needs to be 2. –This risk occurs due to the Immutable nature of Smart Contract: deployed code cannot be modified. Therefore, if the required cryptographic key is lost, there is no mechanism to override the encrypted rules. Smart contracts rely on “cryptographic enforcement,” not “legal enforcement” or any third-party intervention, to restore funds.

Multi-signature contracts, although enhancing security by requiring multiple confirmations, still face the major security challenge of losing money (cannot be rolled back)